

A Concurrent Queue System for Multi-GPU Platforms: Application to Bellman-Ford SSSP

Beyza Çavuşoğlu
Computer Engineering
Koc University
Istanbul, Turkey

bcavusoglu23@ku.edu.tr 0009-0005-9120-3878

Abstract—This paper presents the design and implementation of a multi-GPU concurrent queue system using NVIDIA’s NVSHMEM. The Bellman-Ford algorithm is used as a case study to evaluate the performance of the proposed concurrent FIFO queue, with this multi-GPU implementation being the first known instance of its kind. Experimental results demonstrate that the multi-GPU queue implementation achieves a maximum speedup of 3.92x and an average speedup of 3.04x over the single-GPU baseline on four NVIDIA A100 GPUs. When applied to the Bellman-Ford Single-Source Shortest Path (SSSP) algorithm, the multi-GPU system achieves a maximum speedup of 3.03x and an average speedup of 2.65x compared to the single-GPU implementation, tested on 10 graphs of different kinds taken from the SuiteSparse Matrix Collection.

Index Terms—Concurrent FIFO Queue, Multi-GPU, NVSHMEM, CUDA, Parallel Computing, Bellman-Ford Algorithm, Distributed Queue

I. INTRODUCTION

Efficient queue management is a fundamental challenge in computing, especially in parallel and distributed systems where multiple threads or processes concurrently access shared data structures. Queues are widely used in applications such as task scheduling, event-driven simulations, and graph algorithms, and typically follow FIFO, LIFO, or priority-based policies depending on the application’s logic [12].

With the emergence of general-purpose computing on GPUs, there has been a growing demand not only to adapt these concurrent data structures to massively parallel architectures, but also to leverage GPUs as a necessary computational platform for their scalability. However, parallelizing queue operations on GPUs introduces challenges such as contention, irregular memory access, and synchronization overhead, especially in dynamic and irregular workloads. While there has been significant research on concurrent FIFO queues, implementations specifically designed for GPU architectures remain limited, as they often fail to fully exploit thread-level parallelism due to blocking behavior, inefficient atomic operations, and inadequate support for fine-grained concurrency [26], [6], [18]. To overcome these limitations, this paper adopts the Broker Queue (BQ) - a FIFO, concurrent, lock-free, and linearizable queue specifically designed for GPUs [15].

Despite their significant potential, multi-GPU systems introduce challenges related to synchronization, memory coherence, and communication between GPUs. One key solution

to address these challenges is NVIDIA’s NVSHMEM [20], which facilitates efficient communication and synchronization between GPUs in a multi-GPU setup, allowing for scalable and faster parallel processing. Unlike prior work, which has primarily focused on single-GPU or CPU-based queue implementations, this research explores and demonstrates the efficiency of concurrent queue operations in multi-GPU environments, filling a critical gap in the field [31,32, 33,34,36]. To achieve this, each GPU is assigned its own partition of the queue using a PGAS (Partitioned Global Address Space) model, where local ring buffers and metadata are shared across GPUs, enabling fully parallel enqueue and dequeue operations.

To evaluate the performance of the proposed multi-GPU concurrent queue system, the Bellman-Ford algorithm was selected as a benchmark application due to its naturally parallelizable structure and heavy reliance on queue operations. The Bellman-Ford algorithm, which solves the SSSP problem, is a fundamental algorithm in graph theory. The SSSP problem involves finding the shortest path from a given source vertex to all other vertices in a weighted graph, where the edge weights may be negative [4]. Bellman-Ford is particularly useful because it can handle graphs with negative weight edges, but its performance can become a bottleneck in large-scale graphs, particularly when implemented in parallel environments. Despite its importance, there has been limited research into the application of GPU systems for optimizing the Bellman-Ford algorithm, especially with respect to the concurrent queue systems that manage updates to the vertices during the algorithm’s execution [1, 5, 21, 35].

Overall, this paper makes the following contributions:

- **Multi-GPU Concurrent FIFO Queues:** A novel concurrent FIFO queue system optimized for multi-GPU environments, achieving a maximum speedup of 3.92x and an average speedup of 3.04x over single-GPU implementations on four NVIDIA A100 GPUs.
- **Bellman-Ford SSSP Optimization:** The first implementation of the Bellman-Ford algorithm utilizing multi-GPU concurrent queues, demonstrating significant efficiency improvements. Testing on diverse graphs from the SuiteSparse Matrix Collection shows a maximum speedup of 3.03x and an average speedup of 2.65x compared to single-GPU implementations.

II. BACKGROUND AND RELATED WORK

A. Concurrent Queue Algorithms

FIFO queues process items in the order they were inserted, making them well-suited for work distribution [11], [15]. These queues are categorized as blocking or non-blocking [17]. Blocking algorithms may cause delays if slower processes hinder faster ones, whereas non-blocking algorithms ensure at least one operation completes within bounded time, enhancing concurrency [23, 3, 13, 22, 24].

In multiprocessor systems, blocking algorithms suffer from performance drops due to scheduling delays, cache misses, or page faults. Non-blocking algorithms prevent such bottlenecks, however, they are not a single monolithic category but rather consist of three distinct types: lock-free, wait-free, and obstruction-free. Lock-free algorithms ensure system-wide progress but may allow individual thread starvation. Wait-free algorithms provide stronger guarantees by ensuring every thread completes in bounded steps [2]. Obstruction-free algorithms [14], are the weakest non-blocking type, guaranteeing progress only in the absence of contention. All wait-free algorithms are lock-free, and all lock-free algorithms are obstruction-free, but not vice versa.

B. Linearizability and Fair Ordering

These are crucial for ensuring predictable behavior in concurrent data structures. Linearizability ensures operations appear sequential, maintaining the order of invocation and response events. For example, if one dequeue occurs at timestamp 1 and another at timestamp 2, this order is maintained despite execution timing differences [15].

Fair ordering ensures equitable access to shared resources, preventing starvation. For example, if two operations are enqueued, the first one is always processed before the second, maintaining a predictable and orderly execution sequence. In FIFO queues, it guarantees that operations are processed in their enqueued order, ensuring efficiency and fairness in multi-threaded systems.

C. GPU Queues

GPUs achieve far higher overall throughput than CPUs by dedicating most of their transistor budget to massively parallel data processing, executing thousands of threads concurrently, whereas CPUs optimize for low-latency single-thread performance with larger caches and complex control logic [28]. CUDA threads access multiple memory spaces, per-thread local, shared, cluster shared, global, constant, and texture memory, each optimized for different performance characteristics and visibility levels. Shared and cluster-shared memory enable fast intra-block communication, while global, constant, and texture memory provide grid-wide access with persistence across kernel launches, supporting efficient data reuse and specialized read-only operations.

While this memory hierarchy enables high intra-GPU parallelism, many real-world workloads require scaling beyond a single device, making efficient multi-GPU coordination

essential. Strong scaling is often limited in traditional multi-GPU applications because CPU-driven communication in MPI or OpenSHMEM introduces synchronization overhead and GPU idling [29]. NVSHMEM overcomes these limitations by enabling GPU-initiated, one-sided communication within a Partitioned Global Address Space (PGAS) model, providing a unified memory abstraction across GPUs (Figure 1). This approach reduces CPU-GPU synchronization, supports long-running kernels that overlap computation and communication, and simplifies development by abstracting low-level communication details [20, 30]. Although variables reside in a shared PGAS, updates are initially local to each Processing Element (PE), as illustrated in Figure 1a; however, NVSHMEM's put/get operations allow efficient remote access to these distributed values. Unlike MPI's two-sided communication, which suffers from synchronization and protocol overheads, NVSHMEM's one-sided primitives-mapped to RDMA or NVLink-enable low-latency data movement and fine-grained concurrency, offering superior scalability for large multi-GPU workloads.

While there has been research on concurrent FIFO queues, studies focusing specifically on GPU-based concurrent FIFO queues remain limited [26], [6], [18]. A key limitation is treating threads as isolated units rather than utilizing GPU warps, leading to high contention in CAS (Compare and Swap) operations [6]. The Combined and CAS Loop Queue (CCLQ) addressed this, achieving a 40× speedup over previous GPU-based approaches.

Selecting a FIFO, concurrent, lock-free, and linearizable queue is critical for GPU performance. FIFO ordering ensures that tasks are executed in the precise order of their submission, while concurrency maximizes GPU resource utilization. Lock-free designs prevent bottlenecks, and linearizability guarantees atomic execution. These properties make the BQ an ideal choice for GPU implementation [15]. Unlike traditional lock-free queues that suffer from excessive failed CAS operations, BQ maintains high performance even under contention. Blocking queues, while stable, hinder load balancing in multi-queue setups. BQ overcomes these challenges, optimizing fine-granular parallel work distribution on GPU cores. Performance evaluations confirm BQ as the fastest queue in single-GPU environments [15], and extending it to multi-GPU systems is expected to yield even greater speedups.

D. SSSP Algorithms - Bellman Ford

The shortest-path problem finds the optimal route between two points with applications in routing, transportation, and social networks [16]. SSSP algorithms include BFS for unweighted graphs [9] and Dijkstra's algorithm for non-negative weighted graphs, though the latter is hard to parallelize and cannot handle negative weights [16]. The Bellman-Ford algorithm [4],[10],[19] overcomes these issues by iteratively relaxing edges, supporting negative weights, and detecting cycles. While its time complexity is $O(nm)$, it is more parallelizable [1,7]. While Dijkstra relies on a priority queue, Bellman-Ford can use simpler data structures, such as a FIFO

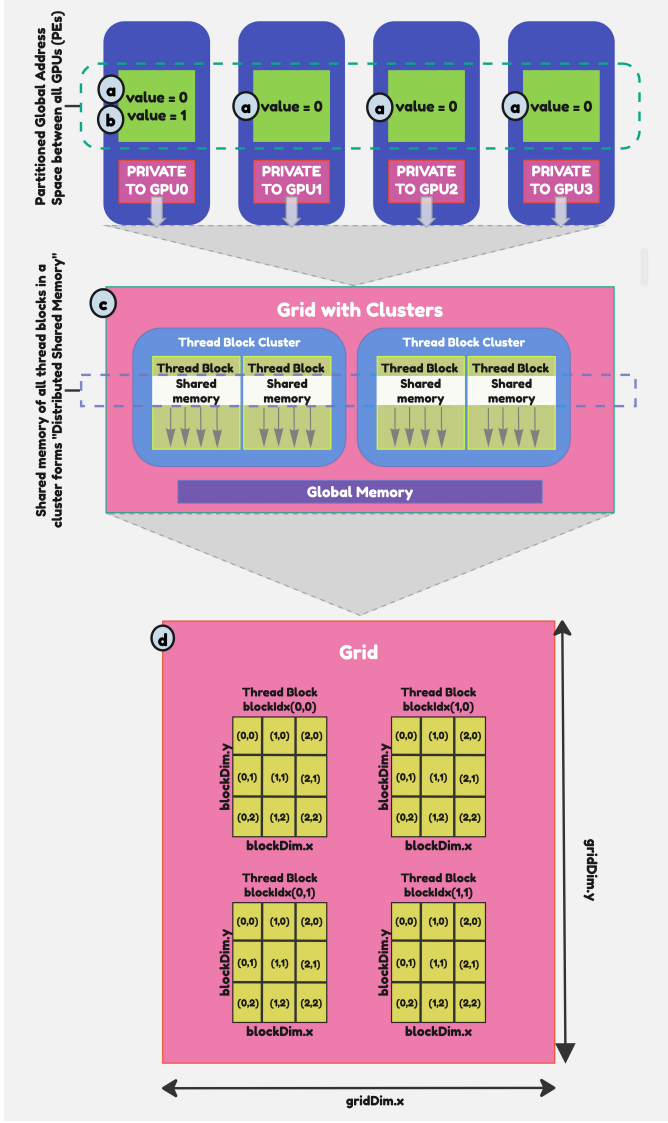


Fig. 1. CUDA and NVSHMEM hierarchy a) value set to 0 in all PEs b) value set to 1 by PE1, only PE 1's memory is updated, the rest of value variables in other PEs stayed as 0. c) GPU memory hierarchy with Thread Block cluster d) grid and block indexing in 2D

queue [8]. This paper explores a parallel FIFO queue-based Bellman-Ford implementation for an efficient SSSP solution.

III. METHODOLOGY AND IMPLEMENTATION

A. Single-GPU Queue Implementation

The BQ is a concurrent data structure for efficient enqueue and dequeue operations [15]. It consists of a ring buffer, a fixed-size circular structure that overwrites old elements when full; head and tail pointers that track enqueue and dequeue positions; a ticket buffer that manages access sequencing to prevent conflicts; and a counter that maintains balance between operations for consistency. This algorithm ensures future operations by tracking count for linearizability and using a ticketing system to prevent race conditions and ensure

fair ordering. The pseudocode of the BQ algorithm is shown in Algorithm 1.

1) *Ensuring*: In traditional queue implementations, threads often skip enqueue operations if the queue is full, a limitation addressed by the broker mechanism in this algorithm. The broker ensures that operations are guaranteed before being executed. To do this, it uses a counter variable, "Count," that prevents modification of the head or tail pointers unless the operation can be completed successfully. For enqueue operations, the `ensureEnqueue` method checks if there is space in the buffer or if items will soon be dequeued. Similarly, `ensureDequeue` checks if an item is available for removal or if new items are being enqueued. This broker mechanism guarantees that only threads with a confirmed operation can modify the queue pointers, preserving the integrity of the queue in concurrent environments and ensuring linearizability.

2) *Ticketing*: To prevent race conditions and ensure orderly access, the algorithm implements a ticketing mechanism. Each ring buffer slot is associated with a ticket, and enqueue and dequeue operations are serialized based on the ticket number. Even-numbered tickets are for enqueue operations, while odd-numbered tickets are for dequeue operations. A thread must obtain a ticket for the slot it wishes to access and wait until its ticket matches the one in the buffer, ensuring controlled, sequential access to the queue and preventing race conditions. This mechanism guarantees fair ordering of operations and maintains the queue's integrity [15].

The "Broker Work Distributor Queue (BWD)" is an alternative to the Broker Queue, designed to simplify the enqueue and dequeue processes [15]. Instead of continuously checking for Full or Empty statuses, the `ensureEnqueue` and `ensureDequeue` functions are only called once during the enqueue and dequeue operations, providing a snapshot of the queue's state. While this design improves efficiency, it comes with a limitation: non-linearizability. The 'Count' variable is used as a temporary assurance but does not always reflect the actual state of the queue. Despite this, the BWD is suitable for work distribution, as it ensures the queue eventually becomes empty unless new items are added. This behavior is sufficient for practical purposes in distributing tasks. Both the BQ and the BWD variants are used in the multi-GPU implementation, with the expectation that BWD will achieve better performance due to its reduced overhead from avoiding repeated state checks.

B. Multi-GPU Queue Implementation

Traditional single-GPU queue implementations typically rely on a centralized structure where all enqueue and dequeue operations are directed to a single ring buffer residing in that GPU's memory. While straightforward, this centralized design becomes a bottleneck under heavy concurrency due to frequent contention on shared variables such as the head and tail pointers. As more threads attempt to simultaneously enqueue or dequeue, serialized access degrades performance, limiting throughput and increasing latency.

Algorithm 1: Single-GPU Broker Queue Algorithm

Function INITIALIZE ():

```
head  $\leftarrow$  0;  
tail  $\leftarrow$  0;  
count  $\leftarrow$  0;  
Tickets[N]  $\leftarrow$  {0, 0, ..., 0};  
RingBuffer[N]  $\leftarrow$  {0, 0, ..., 0};
```

Function WAIT_FOR_TICKET (Pos, Expected):

```
while Ticket[Pos]  $\neq$  Expected do  
    sleep();
```

Function ENSURE_ENQUEUE ():

```
Num  $\leftarrow$  atomicLoad(count);  
ensurance  $\leftarrow$  false;  
while Num < N and !ensurance do  
    if atomicAdd(count, 1) < N then  
        ensurance  $\leftarrow$  true;  
    else  
        Num  $\leftarrow$  atomicSub(count, 1) - 1;  
return ensurance;
```

Function ENSURE_DEQUEUE ():

```
Num  $\leftarrow$  atomicLoad(count);  
ensurance  $\leftarrow$  false;  
while Num > 0 and !ensurance do  
    if atomicAdd(count, -1) <= 0 then  
        atomicAdd(count, 1);  
        Num  $\leftarrow$  atomicLoad(count);  
return ensurance;
```

Function PUT_DATA (data):

```
Pos  $\leftarrow$  atomicAdd(tail, 1);  
P  $\leftarrow$  Pos % N;  
WAIT_FOR_TICKET(P, 2*(Pos/N));  
RingBuffer[P]  $\leftarrow$  data;  
Tickets[P]  $\leftarrow$  2*(Pos/N) + 1;
```

Function READ_DATA (data):

```
Pos  $\leftarrow$  atomicAdd(head, 1);  
P  $\leftarrow$  Pos % N;  
WAIT_FOR_TICKET(P, 2 * (Pos / N) + 1);  
poppedData  $\leftarrow$  RingBuffer[P];  
Tickets[P]  $\leftarrow$  2*(Pos + N/N);
```

Function ENQUEUE (data):

```
while not ENSURE_ENQUEUE() do  
    if N  $\leq$  (tail - head) < N + MaxThreads/2  
        then  
            return Full;  
PUT_DATA(data);  
return Successful;
```

Function DEQUEUE (data):

```
while not ENSURE_DEQUEUE() do  
    if N + MaxThreads/2  $\leq$  (tail - head - 1) then  
        return Empty;  
return READ_DATA(data);
```

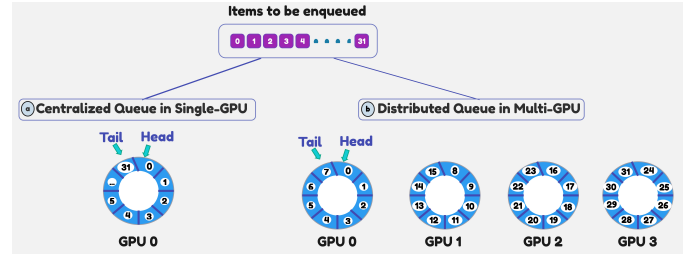


Fig. 2. Illustration of Single-GPU and Multi-GPU scenarios

To overcome these limitations, multi-GPU systems distribute the workload and memory footprint across several GPUs. However, enabling efficient inter-GPU communication and coordination requires a careful memory management strategy. This paper presents a distributed queue implementation across multiple GPUs, where both data and operations are partitioned in a scalable fashion. Instead of centralizing the queue in one device, the global queue is logically divided among GPUs, with each GPU responsible for a local segment of the queue, managed independently but synchronized globally.

The centralized design (Figure 2a) places the entire queue within GPU 0, where all threads perform their operations on a single shared ring buffer. This leads to contention and potential performance degradation as the system scales. In contrast, the distributed design (Figure 2b) assigns each GPU a local queue segment with its own private ring buffer and ticket buffer. Each GPU executes enqueue and dequeue operations over its assigned portion of the global queue, reusing the single-GPU APIs with remote atomic updates to head and tail, significantly reducing contention and enhancing parallelism. The global queue is effectively distributed in a round-robin fashion across GPUs, enabling better utilization of memory and compute resources.

The implementation (Algorithm 2) is built upon the Partitioned Global Address Space (PGAS) model and uses NVSHMEM for efficient memory sharing and synchronization across GPUs. In this model, each PE, corresponding to a GPU, participates in a collective symmetric memory allocation. Specifically, NVSHMEM's `nvshmem_malloc` ensures that each GPU allocates identical-sized regions in its local symmetric heap, resulting in symmetric addresses that are valid and accessible across PEs. This enables any GPU to access another GPU's memory location using a known symmetric address and remote PE ID [20].

In this design, each GPU's queue segment (i.e., its portion of the ring buffer and ticket buffer) is logically private: it is primarily accessed by the GPU that owns it, as observed previously in Figure 1. However, due to symmetric allocation, these regions remain physically accessible by all other GPUs when needed for coordination or global queue management. In contrast, the coordination metadata—particularly the head and tail pointers used to track global enqueue and dequeue progress—is explicitly shared and accessed by all participating GPUs. This allows the system to maintain a coherent global

view of the state of the queue even though the data are partitioned locally.

Algorithm 2: Multi-GPU Broker/BWD Queue System

```

Function MultiGPU_INIT ():
    // Allocate queue in NVSHMEM PGAS
    // instead of local GPU memory
    head ← nvshmem_malloc(sizeof(int));
    tail ← nvshmem_malloc(sizeof(int));
    count ← nvshmem_malloc(sizeof(int));
    tickets[N] ← nvshmem_malloc(N);
    ring[N] ← nvshmem_malloc(N);
    // All GPUs initialize their local
    // copy
    if nvshmem_my_pe() == 0 then
        head ← 0, tail ← 0, count ← 0;
        for i ← 0 to N − 1 do
            tickets[i] ← 0;
            ring[i] ← 0;
    nvshmem_barrier_all();

Function MultiGPU_ENQUEUE (value):
    pe ← nvshmem_my_pe();
    numGPUs ← nvshmem_n_pes();
    // Compute partition range for this
    // GPU
    opsPerGPU ← numOperations/numGPUs;
    start ← pe · opsPerGPU;
    end ← (pe == numGPUs-1) ? numOperations :
    start + opsPerGPU;
    for i ← start + threadId to end by
    totalThreads do
        success ← SINGLE_GPU_ENQUEUE (value,
        pe, count, tail, tickets, ring);
        if success then
            atomicAdd(succesfulEnqueues, 1);

Function MultiGPU_DEQUEUE ():
    pe ← nvshmem_my_pe();
    opsPerGPU, start, end defined as above;
    for i ← start + threadId to end by totalThreads do
        data ← SINGLE_GPU_DEQUEUE (count, pe,
        head, tickets, ring);
        if data ≠ EMPTY then
            atomicAdd(succesfulDequeues, 1);

```

C. Single and Multi-GPU Bellman-Ford

The Single-GPU Bellman-Ford implementation utilizes the aforementioned BQ structure that holds edges for relaxation, enabling concurrent enqueueing and improving efficiency. The algorithm begins by transferring the graph data—represented as Edge structures with source, destination, and weight fields—to GPU memory for parallel processing. The distance array is initialized with the source vertex distance set to zero

and all other vertices set to infinity. The parallel relaxation phase employs multiple threads that concurrently dequeue edges from the BQ and attempt to update vertex distances if shorter paths are found. Each thread performs atomic operations to maintain consistency when multiple threads modify the same vertex’s distance. A change counter is used to track updates during iterations, serving both as a mechanism for negative cycle detection and as an early termination condition. It is important to note that this work does not aim to provide the most optimized single-GPU Bellman-Ford implementation, as achieved in prior studies [27], but rather to evaluate a queue-based design in order to analyze its performance trends in a multi-GPU environment.

The multi-GPU Bellman-Ford (Algorithm 3) partitions the graph and distance array across GPUs using BQ or BWD queues for efficient distributed processing. Each GPU maintains its own broker queue and updates distances for its assigned vertices, reducing the inter-GPU communication latency and avoids bottlenecks from centralized data access. Moreover, to optimize performance, queue management benefits from simpler atomic operations (e.g., ‘atomicAdd’) instead of complex compare-and-swap mechanisms. One-sided NVSHMEM operations (‘put/get’) enable inter-GPU distance updates when needed. A key feature is the implementation of efficient peer-to-peer communication channels between GPUs, allowing for direct memory transfers without CPU intervention. Additionally, the implementation includes a distributed negative cycle detection mechanism that coordinates across all GPUs to identify negative cycles spanning multiple devices.

IV. EXPERIMENTAL RESULTS AND EVALUATION

Tests were conducted on a workstation with four NVLink-connected NVIDIA A100-SXM4-40GB GPUs using CUDA 12.3. The first phase assesses the performance of the single-gpu queue in various configurations: thread counts (32-512), block counts (64, 108, 216), L2 cache set-aside ratios (0-75%), and memory types (unified vs. global) to determine optimal configurations and establish a performance baseline for evaluating multi-GPU speedup. Multi-GPU tests focus on global memory without L2-cache set-aside, under varying thread and block counts. Kernel execution introduces overhead from initialization, and memory setup. To mitigate this, a warm-up phase executes kernels ten times before performance measurements. Moreover, data transfer for enqueue operations is completed beforehand, ensuring that only enqueue and dequeue times are tested. For Single-GPU tests, 10 million operations are performed on the queue in two scenarios:

- 1) **Global Enqueue-Dequeue (EnqDeq)**: Sequential enqueue followed by dequeue operations, isolating their performance.
- 2) **Global Mixed**: Concurrent enqueue and dequeue, with odd-numbered blocks performing dequeue and even-numbered blocks enqueue, simulating real-world workloads.

The second phase evaluates practical performance using the Bellman-Ford. Tests compare single vs. multi-GPU implementations across multiple thread and block variations but only using global memory and no L2 cache set-aside, using 10

Algorithm 3: Multi-GPU Bellman-Ford Algorithm

Input: Graph G with vertices V and edges E , source vertex s , N_GPUs

Output: Shortest distances d from source vertex, detection of negative cycles

Function

```
INITIALIZE_DISTRIBUTED_DISTANCES
( $d\_dist$ ,  $V$ ,  $source$ ):
  foreach vertex index  $i$  assigned to this thread (by
    striding over all vertices) do
    if  $i = source$  then
      Use NVSHMEM to set  $d\_dist[i]$  to 0 on
        this PE;
    else
      Use NVSHMEM to set  $d\_dist[i]$  to  $\infty$  on
        this PE;
```

Function PARALLEL_RELAX_EDGES ($Graph\ G$, GPU_ID):

Partition the edge list so each GPU gets a contiguous chunk of edges and stores in its own BQ or BWD;

foreach edge assigned to this thread in its GPU's chunk **do**

```
   $owner1 \leftarrow edge.v1 \bmod N\_GPUs$ ;
   $owner2 \leftarrow edge.v2 \bmod N\_GPUs$ ;
   $dist1 \leftarrow nvshmem\_atomic\_fetch($ 
     $d\_dist[edge.v1], owner1)$ 
   $dist2 \leftarrow nvshmem\_atomic\_fetch($ 
     $d\_dist[edge.v2], owner2)$ 
  if  $dist1 \neq \infty$  then
    if  $dist1 + edge.weight < dist2$  then
       $new\_dist \leftarrow dist1 + edge.weight$ ;
       $nvshmem\_put(d\_dist[edge.v2],$ 
         $new\_dist, owner2)$ 
      Atomically increment changes;
```

Function MULTI_GPU_BELLMANFORD ($Graph\ G$, $Source\ s$, N):

Allocate $head, tail, count, ring_buffer, tickets$ in PGAS using $nvshmem_malloc$;

INITIALIZE_DISTRIBUTED_DISTANCES (G ,

s , GPU_ID);

$hasNegativeCycle \leftarrow false$;

$changes \leftarrow$ shared atomic counter;

for $int\ i = 0$ to V **do**

```
  PARALLEL_RELAX_EDGES ( $G$ ,  $GPU\_ID$ );
   $totalChanges \leftarrow reduce\_sum(changes)$  across
    all GPUs;
  if  $i == V - 1 \wedge totalChanges > 0$  then
     $hasNegativeCycle \leftarrow true$ ;
```

// Gather final distances from distributed memory

for vertex $v = 0$ to $V - 1$ **do**

```
   $ownerPE \leftarrow v \bmod N\_GPUs$ ;
   $d[v] \leftarrow nvshmem\_get(d\_dist[v], ownerPE)$ ;
```

return $d, hasNegativeCycle$

SuiteSparse graphs [25]. Speedup is discussed when using up to 4 GPUs.

A. Single and Multi-GPU Queue Implementation Results

Both queue designs, in both single and multi-GPU configurations demonstrate strong scalability as thread and block counts increase (Figure 3 and 4), particularly between 32 and 256 threads, showing significant reductions in total execution time which comes from the ability to support highly concurrent enqueue-dequeue operations, enabling efficient parallelism. However beyond 256 threads, performance gains begin to plateau, indicating that the system approaches its saturation point for concurrency. Additionally, performance under Unified Memory is less stable than Global Memory, especially under high-concurrency conditions, which is expected given that Unified Memory generally exhibits higher-latency data access patterns, less predictable and is still undergoing performance refinement by NVIDIA [28].

In mixed-workload scenarios, the Broker Queue maintains high efficiency, exhibiting decreasing execution times as concurrency increases and effectively balancing enqueue and dequeue operations. The BWD Queue also functions correctly; however, its throughput remains lower than the Broker's under mixed conditions. This difference arises from BWD's non-linearizable design, where ensureEnqueue and ensureDequeue are invoked only once per operation rather than being continuously evaluated. As the queue state changes rapidly when enqueues and dequeues interleave, this single-shot check may not fully capture the current contention level, occasionally resulting in unnecessary rejections even when capacity becomes available shortly after. In contrast, the Broker continuously re-validates these conditions, enabling it to adapt to dynamic state changes and sustain higher parallelism. Consequently, while both queues behave predictably and scale with increasing concurrency, the Broker achieves superior performance in mixed workloads, highlighting how synchronization design and state accuracy directly influence throughput under highly interleaved access patterns.

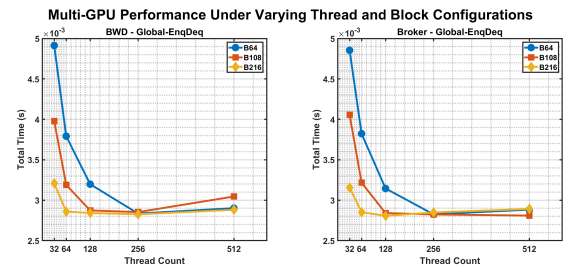


Fig. 3. Multi-GPU Broker and BWD Queues under different thread and block counts

Figure 5 shows the speedup achieved by the Broker and BWD queues when transitioning from single-GPU to multi-GPU configurations. Both queues show diminishing speedup as thread counts increase, likely due to single-GPU efficiency and multi-GPU overhead. The maximum speedup for Broker

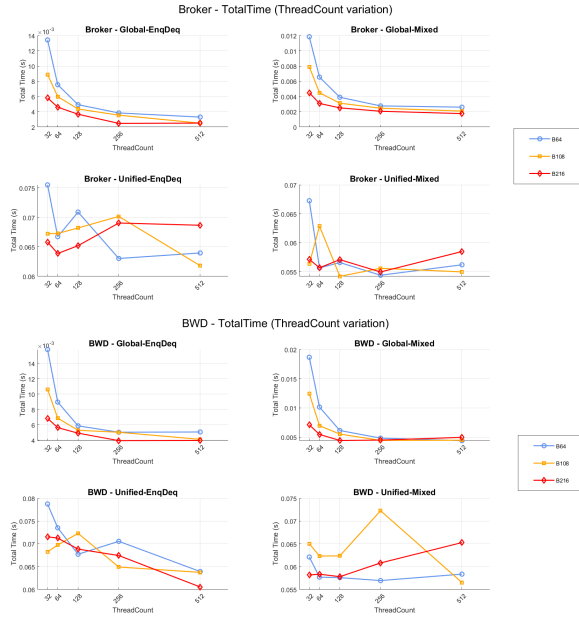


Fig. 4. (a) Single-GPU Broker Queue Unified and Global Memory under different thread and block counts, (b) Single-GPU BWD Queue Unified and Global Memory under different thread and block counts

Queue is 3.92x (average 3.04) and for BWD Queue, it's 3.88 (average 3.02). With 2 GPUs, the maximum speedup is 1.42x (average 1.16), indicating performance may degrade with fewer GPUs.

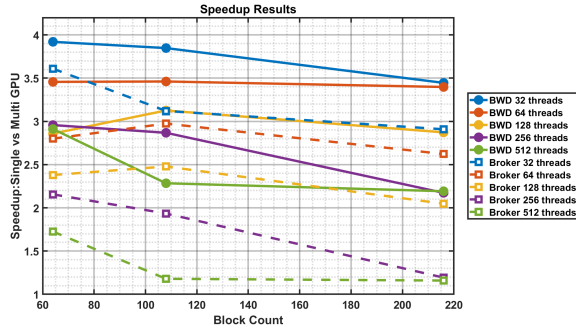


Fig. 5. Speedup with 4 GPUs: Single-GPU vs Multi-GPU Broker and BWD Queues under different thread and block counts

B. Single and Multi-GPU Bellman-Ford Implementation Results

For single and multi-GPU implementations, both queues show an increase in execution time at higher thread counts, especially at 128 and 256 (Figures 6 and 7). Smaller block sizes (64) consistently show the lowest execution times, suggesting better resource utilization. However, as thread counts increase, performance diminishes due to lower occupancy, memory bandwidth saturation. Additionally, kernel launch overhead becomes significant when thread counts are high relative to block counts, diminishing parallelism benefits. Resource contention, especially in atomic operations, introduces serialization, further reducing efficiency. The Bellman-Ford

algorithm's inherent dependencies also limit scalability, and as more threads are added, uneven workload distribution may lead to idle threads.

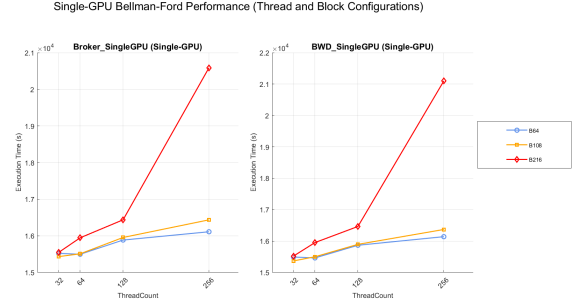


Fig. 6. Single-GPU BF for Broker and BWD Queues under different thread and block counts

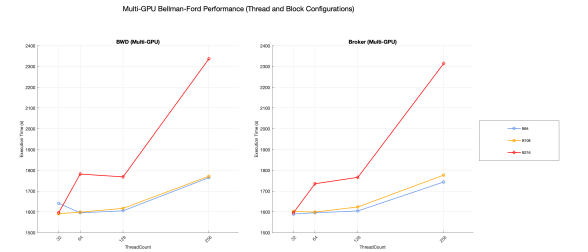


Fig. 7. Multi-GPU BF for Broker and BWD Queues under different thread and block counts

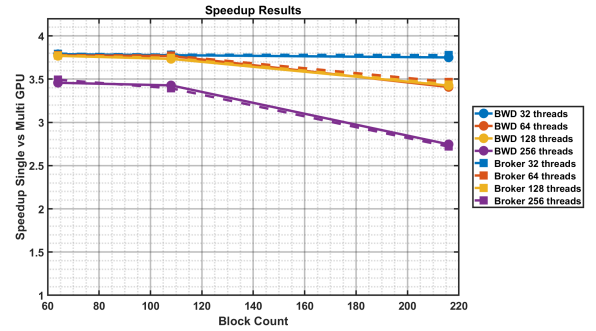


Fig. 8. Speedup over 4 GPUs: BF of Single-GPU vs Multi-GPU Broker and BWD Queues under different thread and block counts for SuiteSparse graph 'pcb3000'

The speedup in Figure 8 compares the Broker and BWD queues for the Bellman-Ford algorithm with single and multi-GPU setups. Both queues show a decline in speedup as the block count increases, with better performance at 64 and 108 blocks, but worse at 216 due to overhead. For the 10 SuiteSparse graphs, the Broker queue achieves a maximum speedup of 3.03 and an average speedup of 2.65. Although Figure 5 shows speedups up to 3.92x, additional algorithmic overheads of Bellman-Ford might be limiting the gains.

Table 1 shows that with SuiteSparse graphs, the algorithm achieved important speedups, likely due to their complex

connectivity, which improves data access patterns and reduces contention in the Bellman-Ford. The best result, from the pcb3000 graph (a linear programming problem), achieved a maximum speedup of 3.79 $\times$ and an average of 3.57 $\times$, demonstrating the benefits of well-structured graph data.

TABLE I
SUITESPARSE GRAPHS [25] AND SINGLE VS. MULTI-GPU SPEEDUP WITH 4 GPUS

Graph	Kind	Vertices	Edges	Max Speedup	Avg. Speedup
pcb3000	Linear prog.	3960	7732	3.79	3.57
G65	Random matrix	8000	8000	3.70	3.48
G67	Random matrix	10000	10000	3.18	3.01
lp_ken_11	Linear prog.	14694	21349	3.10	2.96
ch7-6-b5	Combinatorial prob.	5040	15120	2.99	2.45
Alemдар	2D/3D prob.	6245	6245	2.32	2.21
Reuters911	Network	13332	13332	2.32	2.23
Foldoc	Net., computing	13356	13356	2.32	2.23
Cyl6	Structural prob.	13681	13681	2.31	2.23
G39	Random matrix	2000	2000	2.29	2.14
For all graphs	—	—	—	3.03	2.65

V. CONCLUSION

This paper presents a multi-GPU concurrent FIFO queue system for graph processing workloads, extending single-GPU structures to improve efficiency in parallel processing. Performance evaluations show a maximum speedup of 3.92 $\times$ and an average of 3.04 $\times$ on four NVIDIA A100 GPUs, enhancing throughput and reducing latency. The Bellman-Ford algorithm, implemented with this novel queue, represents the first exploration of optimizing this algorithm using concurrent queues in a multi-GPU context. It achieves a maximum speedup of 3.03 $\times$ and an average of 2.65 $\times$, demonstrating its effectiveness for graph processing.

REFERENCES

- [1] P. Agarwal and M. Dutta, "New approach of Bellman-Ford algorithm on GPU using compute unified design architecture (CUDA)," *International Journal of Computer Applications*, vol. 110, no. 13, pp. 975–8887, 2015.
- [2] A. Williams, "Safety: off - How not to shoot yourself in the foot with C++ atomics," 2015. [Online]. Available: https://www.justsoftwaresolutions.co.uk/files/safety_off.pdf
- [3] G. Barnes, "A method for implementing lock-free shared-data structures," in *Proc. 5th ACM Symp. on Parallel Algorithms and Architectures*, pp. 261–270, 1993.
- [4] R. Bellman, "On a routing problem," *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.
- [5] F. Busato and N. Bombieri, "An efficient implementation of the Bellman-Ford algorithm for Kepler GPU architectures," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 8, pp. 2222–2233, 2015.
- [6] D. Cederman, B. Chatterjee, and P. Tsigas, "Understanding the performance of concurrent data structures on graphics processors," in *Euro-Par 2012*, pp. 883–894, Springer, 2012.
- [7] V. T. Chakaravarthy, F. Checconi, P. Murali, F. Petrini, and Y. Sabharwal, "Scalable single source shortest path algorithms for massively parallel systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 7, pp. 2031–2045, 2016.
- [8] M. Chen, R. A. Chowdhury, V. Ramachandran, D. L. Roche, and L. Tong, "Priority queues and Dijkstra's algorithm," Univ. of Texas at Austin, Tech. Rep., 2007.
- [9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- [10] L. R. Ford, "Network flow theory," *Rand Corporation Paper*, Santa Monica, 1956.
- [11] A. Gottlieb, B. D. Lubachevsky, and L. Rudolph, "Basic techniques for the efficient coordination of very large numbers of cooperating sequential processors," *ACM Trans. Program. Lang. Syst.*, vol. 5, no. 2, pp. 164–189, 1983.
- [12] A. Haas, C. M. Kirsch, M. Lippautz, and H. Payer, "How FIFO is your concurrent FIFO queue?," in *Proc. ACM Workshop on Relaxed Synchronization (RS) for Multicore*, pp. 1–8, 2012.
- [13] M. Herlihy, "A methodology for implementing highly concurrent data objects," *ACM Trans. Program. Lang. Syst.*, vol. 15, no. 5, pp. 745–770, 1993.
- [14] M. Herlihy, V. Luchangco, and M. Moir, "Obstruction-free synchronization: Double-ended queues as an example," in *Proc. 23rd Int. Conf. on Distributed Computing Systems*, pp. 522–529, IEEE, 2003.
- [15] B. Kerbl, M. Kenzel, J. H. Mueller, D. Schmalstieg, and M. Steinberger, "The broker queue: A fast, linearizable FIFO queue for fine-granular work distribution on the GPU," in *Proc. Int. Conf. on Supercomputing*, pp. 76–85, 2018.
- [16] A. Madkour, W. G. Aref, F. U. Rehman, M. A. Rahman, and S. Basalamah, "A survey of shortest-path algorithms," *arXiv preprint arXiv:1705.02044*, 2017.
- [17] M. M. Michael and M. L. Scott, "Simple, fast, and practical non-blocking and blocking concurrent queue algorithms," in *Proc. 15th ACM Symp. on Principles of Distributed Computing*, pp. 267–275, 1996.
- [18] P. Misra and M. Chaudhuri, "Performance evaluation of concurrent lock-free data structures on GPUs," in *Proc. IEEE 18th Int. Conf. on Parallel and Distributed Systems*, pp. 53–60, 2012.
- [19] E. F. Moore, "The shortest path through a maze," in *Proc. Int. Symp. on the Theory of Switching*, pp. 285–292, Harvard Univ. Press, 1959.
- [20] NVIDIA, "NVSHMEM Guide," 2024. [Online]. Available: <https://docs.nvidia.com/nvshmem/api/index.html/>
- [21] Z. Hu, J. Sun, L. Jiang, Y. Wang, J. Lin, Y. Zong, and G. Sun, "Beyond a single queue: Multi-level-multi-queue as an effective design for SSSP problems on GPUs," 2026.doi: 10.48550/arXiv.2602.10080.
- [22] S. Prakash, Y.-H. Lee, and T. Johnson, "A nonblocking algorithm for shared queues using compare-and-swap," *IEEE Trans. Comput.*, vol. 43, no. 5, pp. 548–559, 1994.
- [23] J. M. Stone, "A simple and correct shared-queue algorithm using compare-and-swap," in *Proc. ACM/IEEE Conf. on Supercomputing*, pp. 495–504, IEEE, 1990.
- [24] J. M. Stone, "A non-blocking compare-and-swap algorithm for a shared circular queue," in *Parallel and Distributed Computing in Engineering Systems*, S. Tzafestas et al., Eds., pp. 147–152, 1992.
- [25] SuiteSparse, "SuiteSparse Matrix Collection," 2024. [Online]. Available: <https://sparse.tamu.edu/>
- [26] X. Zhang, Y. Deng, and S. Mu, "Toward concurrent lock-free queues on GPUs," *IEICE Trans. Inf. Syst.*, vol. 97, no. 7, pp. 1901–1904, 2014.
- [27] Y. Zhang, A. Brahmakshatriya, X. Chen, L. Dhulipala, S. Kamil, S. Amarasinghe, and J. Shun, "Optimizing ordered graph algorithms with GraphLT," in *Proc. 18th ACM/IEEE Int'l Symp. on Code Generation and Optimization (CGO)*, pp. 158–170, 2020.
- [28] NVIDIA, "CUDA C Programming Guide," 2024. [Online]. https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf.
- [29] MPI, "MPI Guide," 2023. [Online]. Available: <https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf/>
- [30] C.-H. Hsu, N. Imam, A. Langer, S. Potluri, and C. J. Newburn, "An initial assessment of NVSHMEM for high performance computing," in *Proc. IEEE Int'l Parallel and Distributed Processing Symp. Workshops (IPDPSW)*, pp. 1–10, 2020.
- [31] K. Dragicevic and D. Bauer, "A survey of concurrent priority queue algorithms," in *Proc. IEEE Int'l Symp. on Parallel and Distributed Processing (IPDPS)*, pp. 1–6, 2008.
- [32] K. Berney, J. Iacono, B. Karsin, and N. Sitchinava, "A parallel priority queue with fast updates for GPU architectures," 2019.
- [33] Y. Chen, F. Hua, Y. Jin, and E. Z. Zhang, "BGPQ: A heap-based priority queue design for GPUs," 2019.
- [34] A. Braginsky, N. Cohen, and E. Petrank, "CBPQ: High-performance lock-free priority queue," in *(Euro-Par)*, pp. 460–474, 2016.
- [35] D. R. Cholla, R. R. K., P. J. Reddy, A. Vajpai, and A. Kumar, "Optimizing the Bellman-Ford algorithm using GPU parallelization," doi: 10.55041/ijssrem47660.
- [36] M. S. H. Polak, D. A. Troendle, and B. Jang, "Boundary-aware concurrent queue: A fast and scalable concurrent FIFO queue on GPU environments," *Applied Sciences*, vol. 15, no. 4, p. 1834, 2025. doi: 10.3390/app15041834.